

→ what if # data elements > # elements in vector register?

→ Idea: break loops s.t. each iteration operates on # elements in vector register.

→ Example: 512 data el., 64-element VREGs → 8 iterations with VLEN=64, one with VLEN=16

→ that is called vector stripmining

→ what if vector data is not strictly, but irregular?

→ Idea: use indirection to combine/pack elements into vector regs. → scatter/gather operations, these also help avoid useless computation on sparse vectors → these are often implemented in HW to handle matrices and indirect indexing

Want to vectorize loops with indirect accesses:

```
for (i=0; i<N; i++)
    A[i] = B[i] + C[D[i]]
```

→ vector loads/stores use index vector which is added to the base reg. to generate addresses

Scatter example

Index Vector	Data Vector (to Store)	Stored Vector (in Memory)
0	3.14	Base+0 3.14
2	6.5	Base+1 X
6	71.2	Base+2 6.5
7	2.71	Base+3 X
		Base+4 X
		Base+5 X
		Base+6 71.2
		Base+7 2.71

Indexed load instruction (Gather)

```
LV vD, rD      # Load indices in D vector
LVI vC, rC, vD  # Load indirect from rC base
LV vB, rB       # Load B vector
ADDV.D vA, vB, vC # Do add
SV vA, rA       # Store result
```

→ what if some ops. should not be executed on a vector? use masked operations, which are a form of predicated execution

```
VLD V0 = A      loop: for (i=0; i<N; i++)
VLD V1 = B        if (a[i] != 0) then b[i]=a[i]*b[i]
VMASK = (V0 != 0)
VMUL V1 = V0 * V1
VST B = V1
```

→ other issues:

→ Stride and bank count have to be relatively prime to one another and there have to be enough banks to cover latency → only then can we sustain 1 element/cycle throughput

→ Storage format of matrix:

→ Row major: consecutive elements in a row are consecutive in mem.

→ Column major: analogous for columns

→ we need to change stride when accessing row vs. column

→ in the example, A has a stride of 1, B has a stride of 10

⇒ bank conflict possible, how do we minimize?

→ more banks, more ports per bank, better data layout for access pattern/

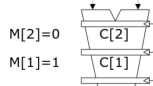
better mapping of address to bank (may be random)

Masked Vector Instructions

Simple Implementation

- execute all N operations, turn off result writeback according to mask

```
M[7]=1 A[7] B[7]
M[6]=0 A[6] B[6]
M[5]=1 A[5] B[5]
M[4]=1 A[4] B[4]
M[3]=0 A[3] B[3]
```

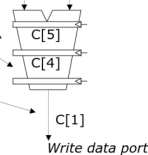


M[2]=0
M[1]=1
M[0]=0
Write Enable
Write data port

Density-Time Implementation

- scan mask vector and only execute elements with non-zero masks

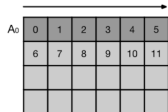
```
M[7]=1
M[6]=0
M[5]=1
M[4]=1
M[3]=0
M[2]=0
M[1]=1
M[0]=0
```



Which one is better?

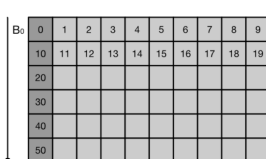
Tradeoffs?

- A and B matrices, both stored in memory in row-major order

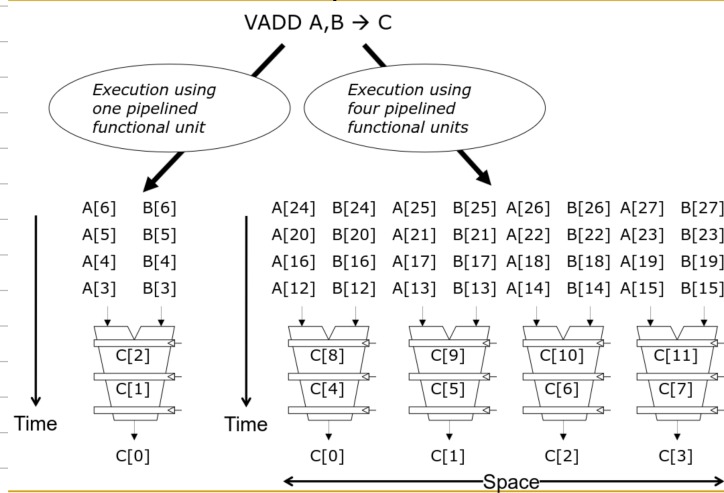


$A_{4 \times 6} B_{6 \times 10} \rightarrow C_{4 \times 10}$

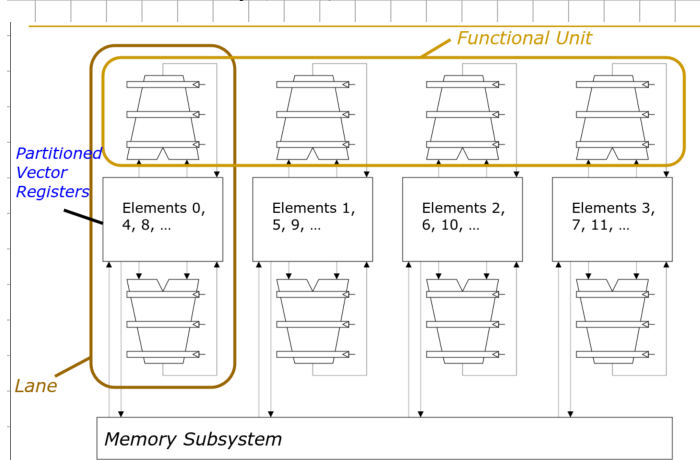
Dot product of each row vector of A with each column vector of B



Vector ins. execution:

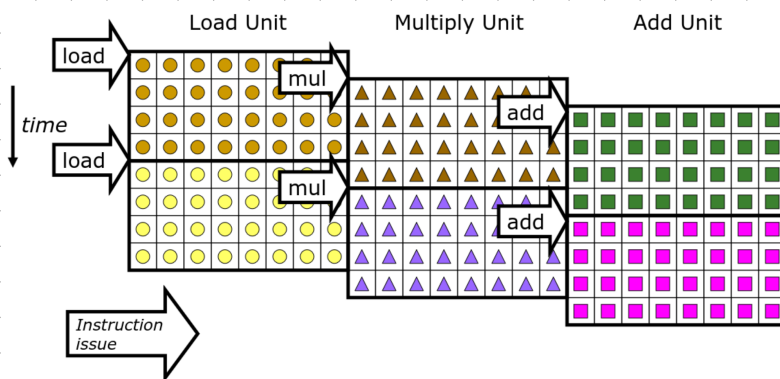


Vector unit structure:

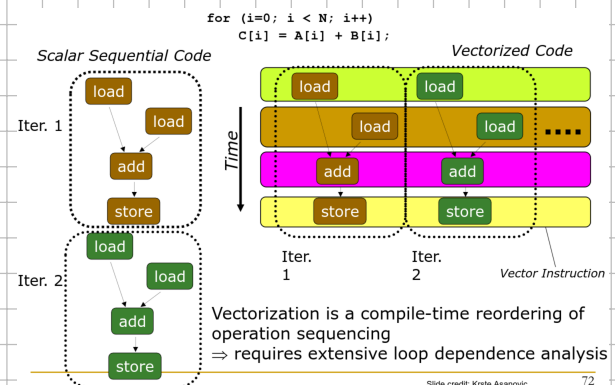


→ Vector instruction level parallelism: execution of multiple vector ins. can overlap

→ Example machine has 32 elements per vector reg. and 8 lanes, example has ^(steady state) 24 ops./cycle and issues one vector ins./cycle



Automatic Code Vectorization :



→ Vector/SIMD machines exploit regular data-level parallelism

→ same op., different elements, → improve performance / simplify design

→ performance improvement limited by vectorizability of code

→ scalar ops. limit vector machine performance (Anzahl)

→ Many current ISAs include vector-like SIMD operations - packed arithmetic performs short op. on multiple inputs

→ example: add 4 8-bit numbers (requires ALU modification to remove carries between 8-bit values)

```
padd8 $s2, $s0, $s1
```

